copying the even elements of f_j into the first $N/2$ locations, and the odd elements into the next $N/2$ elements in reverse order. However, it is not easy to implement the alternative algorithm without a temporary storage array and we prefer the above in-place algorithm.

Finally, we mention that there exist fast cosine transforms for small N that do not rely on an auxiliary function or use an FFT routine. Instead, they carry out the transform directly, often coded in hardware for fixed N of small dimension [1].

CITED REFERENCES AND FURTHER READING:

Walker, J.S. 1996, *Fast Fourier Transforms*, 2nd ed. (Boca Raton, FL: CRC Press)

Rao, K.R. and Yip, P. 1990, *Discrete Cosine Transform: Algorithms, Advantages, Applications* (San Diego, CA: Academic Press)

Hou, H.S. 1987, "A Fast, Recursive Algorithm for Computing the Discrete Cosine Transform," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. ASSP-35, pp. 1455–1461 [see for additional references].

Chen, W., Smith, C.H., and Fralick, S.C. 1977, "A Fast Computational Algorithm for the Discrete Cosine Transform," *IEEE Transactions on Communications*, vol. COM-25, pp. 1004–1009.[1]

12.5 FFT in Two or More Dimensions

Given a complex function $h(k_1, k_2)$ defined over the two-dimensional grid $0 \le k_1 \le N_1 - 1,\ 0 \le k_2 \le N_2 - 1$, we can define its two-dimensional discrete Fourier transform as a complex function $H(n_1, n_2)$, defined over the same grid,

$$H(n_1, n_2) \equiv \sum_{k_2=0}^{N_2-1} \sum_{k_1=0}^{N_1-1} \exp(2\pi i k_2 n_2/N_2)\ \exp(2\pi i k_1 n_1/N_1)\ h(k_1, k_2) \tag{12.5.1}$$

By pulling the "subscripts 2" exponential outside of the sum over k_1, or by reversing the order of summation and pulling the "subscripts 1" outside of the sum over k_2, we can see instantly that the two-dimensional FFT can be computed by taking one-dimensional FFTs sequentially on each index of the original function. Symbolically,

$$\begin{aligned} H(n_1, n_2) &= \text{FFT-on-index-1 (FFT-on-index-2 } [h(k_1, k_2)]) \\ &= \text{FFT-on-index-2 (FFT-on-index-1 } [h(k_1, k_2)]) \end{aligned} \tag{12.5.2}$$

For this to be practical, of course, both N_1 and N_2 should be some efficient length for an FFT, usually a power of 2. Programming a two-dimensional FFT, using (12.5.2) with a one-dimensional FFT routine, is a bit clumsier than it seems at first. Because the one-dimensional routine requires that its input be in consecutive order as a one-dimensional complex array, you find that you are endlessly copying things out of the multidimensional input array and then copying things back into it. This is not recommended technique. Rather, you should use a multidimensional FFT routine, such as the one we give below.

The generalization of (12.5.1) to more than two dimensions, say to L dimensions, is evidently

$$H(n_1,\ldots,n_L) \equiv \sum_{k_L=0}^{N_L-1} \cdots \sum_{k_1=0}^{N_1-1} \exp(2\pi i k_L n_L/N_L) \times \cdots \times \exp(2\pi i k_1 n_1/N_1)\, h(k_1,\ldots,k_L) \tag{12.5.3}$$

where n_1 and k_1 range from 0 to $N_1-1,\ldots$, and n_L and k_L range from 0 to N_L-1. How many calls to a one-dimensional FFT are in (12.5.3)? Quite a few! For each value of $k_1,k_2,\ldots,k_{L-1}$ you FFT to transform the L index. Then for each value of $k_1,k_2,\ldots,k_{L-2}$ and n_L you FFT to transform the $L-1$ index. And so on. It is best to rely on someone else having done the bookkeeping for once and for all.

The inverse transforms of (12.5.1) or (12.5.3) are just what you would expect them to be: Change the i's in the exponentials to $-i$'s, and put an overall factor of $1/(N_1\times\cdots\times N_L)$ in front of the whole thing. Most other features of multidimensional FFTs are also analogous to features already discussed in the one-dimensional case:

- Frequencies are arranged in wraparound order in the transform, but now for each separate dimension.
- The input data are also treated as if they were wrapped around. If they are discontinuous across this periodic identification (in any dimension), then the spectrum will have some excess power at high frequencies because of the discontinuity. The fix, if you care, is to remove multidimensional linear trends.
- If you are doing spatial filtering and are worried about wraparound effects, then you need to zero-pad all around the border of the multidimensional array. However, be sure to notice how costly zero-padding is in multidimensional transforms. If you use too thick a zero-pad, you are going to waste a *lot* of storage, especially in three or more dimensions!
- Aliasing occurs as always if sufficient bandwidth limiting does not exist along one or more of the dimensions of the transform.

The routine `fourn` that we furnish herewith is a descendant of one written by N.M. Brenner. It requires as input (i) a vector, telling the length of the array in each dimension, e.g., $(32, 64)$ (note that these lengths *must all* be powers of 2, and are the numbers of *complex* values in each direction); (ii) the usual scalar equal to ± 1 indicating whether you want the transform or its inverse; and, finally, (iii) the array of data. The number of dimensions is determined from the length of the vector in (i).

A few words about the data array: `fourn` accesses it as a one-dimensional array of real numbers, that is, `data[0..(2N1N2...NL)-1]`, of length equal to twice the product of the lengths of the L dimensions. It assumes that the array represents an L-dimensional complex array, with individual components ordered as follows: (i) each complex value occupies two sequential locations, a real part followed by an imaginary; (ii) the first subscript changes least rapidly as one goes through the array; the last subscript changes most rapidly (that is, "store by rows," the C++ norm). Figure 12.5.1 illustrates the format of the output array.

As for `four1` earlier, we find it useful to give a bare form of the routine, where the data array is passed as a pointer, and then an overloaded function that passes the data array (by reference) as a `VecDoub`.

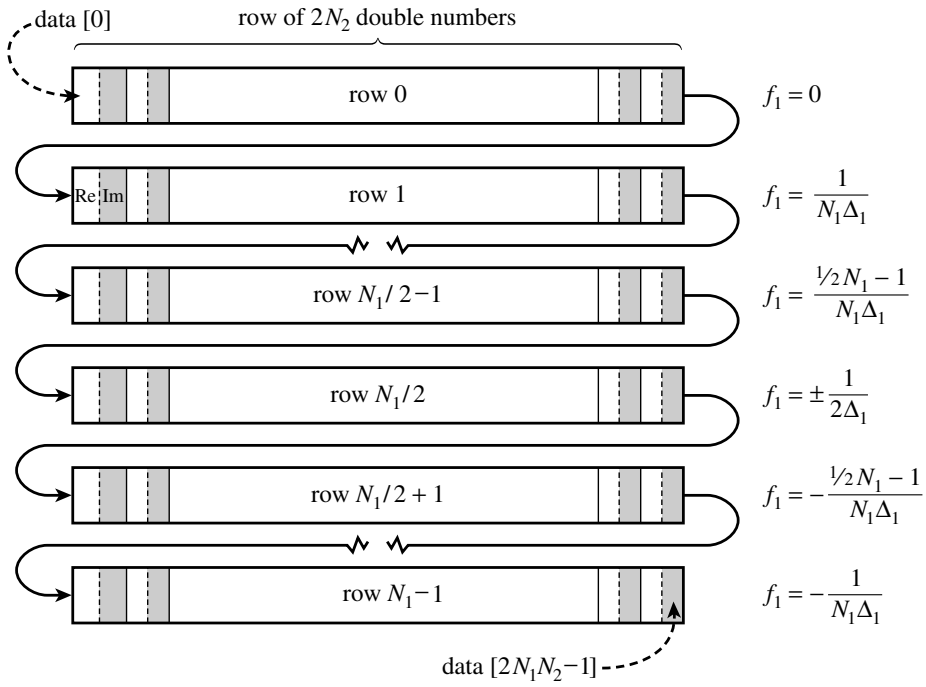

Figure 12.5.1. Storage arrangement of frequencies in the output $H(f_1, f_2)$ of a two-dimensional FFT. The input data is a two-dimensional $N_1 \times N_2$ array $h(t_1, t_2)$ (stored by columns of complex numbers). The output is also stored by complex columns. Each column corresponds to a particular value of f_2, as shown in the figure. Within each column, the arrangement of frequencies f_1 is exactly as shown in Figure 12.2.2. Δ_1 and Δ_2 are the sampling intervals in the 1 and 2 directions, respectively. The total number of (real) array elements is $2N_1N_2$. The program `fourn` can also do more than two dimensions, and the storage arrangement generalizes in the obvious way.

```
void fourn(Doub *data, VecInt_I &nn, const Int isign) {
```
fourier_ndim.h

Replaces `data` by its `ndim`-dimensional discrete Fourier transform, if `isign` is input as 1. `nn[0..ndim-1]` is an integer array containing the lengths of each dimension (number of complex values), which must all be powers of 2. `data` is a real array of length twice the product of these lengths, in which the data are stored as in a multidimensional complex array: real and imaginary parts of each element are in consecutive locations, and the rightmost index of the array increases most rapidly as one proceeds along `data`. For a two-dimensional array, this is equivalent to storing the array by rows. If `isign` is input as -1, `data` is replaced by its inverse transform times the product of the lengths of all dimensions.

```
    Int idim,i1,i2,i3,i2rev,i3rev,ip1,ip2,ip3,ifp1,ifp2;
    Int ibit,k1,k2,n,nprev,nrem,ntot=1,ndim=nn.size();
    Doub tempi,tempr,theta,wi,wpi,wpr,wr,wtemp;
    for (idim=0;idim<ndim;idim++) ntot *= nn[idim];  Total no. of complex values.
    if (ntot<2 || ntot&(ntot-1)) throw("must have powers of 2 in fourn");
    nprev=1;
    for (idim=ndim-1;idim>=0;idim--) {                Main loop over the dimensions.
        n=nn[idim];
        nrem=ntot/(n*nprev);
        ip1=nprev << 1;
        ip2=ip1*n;
        ip3=ip2*nrem;
        i2rev=0;
        for (i2=0;i2<ip2;i2+=ip1) {                   This is the bit-reversal sec-
            if (i2 < i2rev) {                         tion of the routine.
```

```
                for (i1=i2;i1<i2+ip1-1;i1+=2) {
                    for (i3=i1;i3<ip3;i3+=ip2) {
                        i3rev=i2rev+i3-i2;
                        SWAP(data[i3],data[i3rev]);
                        SWAP(data[i3+1],data[i3rev+1]);
                    }
                }
            }
            ibit=ip2 >> 1;
            while (ibit >= ip1 && i2rev+1 > ibit) {
                i2rev -= ibit;
                ibit >>= 1;
            }
            i2rev += ibit;
        }
        ifp1=ip1;                                   Here begins the Danielson-
        while (ifp1 < ip2) {                            Lanczos section of the
            ifp2=ifp1 << 1;                             routine.
            theta=isign*6.28318530717959/(ifp2/ip1);    Initialize for the trigonomet-
            wtemp=sin(0.5*theta);                       ric recurrence.
            wpr= -2.0*wtemp*wtemp;
            wpi=sin(theta);
            wr=1.0;
            wi=0.0;
            for (i3=0;i3<ifp1;i3+=ip1) {
                for (i1=i3;i1<i3+ip1-1;i1+=2) {
                    for (i2=i1;i2<ip3;i2+=ifp2) {
                        k1=i2;                          Danielson-Lanczos formula:
                        k2=k1+ifp1;
                        tempr=wr*data[k2]-wi*data[k2+1];
                        tempi=wr*data[k2+1]+wi*data[k2];
                        data[k2]=data[k1]-tempr;
                        data[k2+1]=data[k1+1]-tempi;
                        data[k1] += tempr;
                        data[k1+1] += tempi;
                    }
                }
                wr=(wtemp=wr)*wpr-wi*wpi+wr;            Trigonometric recurrence.
                wi=wi*wpr+wtemp*wpi+wi;
            }
            ifp1=ifp2;
        }
        nprev *= n;
    }
}

void fourn(VecDoub_IO &data, VecInt_I &nn, const Int isign) {
Overloaded version for the case where data is of type VecDoub.
    fourn(&data[0],nn,isign);
}
```

CITED REFERENCES AND FURTHER READING:

Nussbaumer, H.J. 1982, *Fast Fourier Transform and Convolution Algorithms* (New York: Springer).